

Reimagining Data Modeling on the Lakehouse: Introducing Vibe Data Modeling

Describe your business in plain English. Get a production-ready Silver-layer data model on Databricks. In hours, not months.

by Amr Ali

Summary

- **Vibe Data Modeling is now available.** It is a Databricks-native, LLM-powered agent that produces the analytical Silver-layer model directly from a plain-English description of your business.
- **From prompt to deployed model in hours**, replacing the six-to-thirty-six-month projects that hand-built Silver models or trimmed generic industry templates have historically required.
- **Iterate the model in natural language.** Every "vibe" produces a new versioned model, validated against more than two hundred enforceable quality rules, reviewed by principal-architect and SME LLM personas, and redeployed to Unity Catalog. No version is overwritten. Every iteration is auditable and reversible.
- **One logical model, many physical layouts.** The agent separates the logical model from its physical deployment. Render the same model as one catalog with domain schemas, a catalog per division, or a catalog per domain. No rebuild required.



Vibe Data Modeling targets the Silver layer of the medallion architecture: Bronze holds raw events, Silver holds the conformed analytical model, Gold holds derived KPIs and dashboards.

The problem with Data Modelling

In every analytics stack, Silver is where they make it or break it. It is the layer every analyst, every data scientist, and every BI tool reads from. If Silver is messy, ungoverned, or full of duplicates, every dashboard and every model above it gets harder, slower, and more expensive. The trouble has always been getting there. Most organizations either spend six months to three

years hand-building a Silver model from scratch, or they buy a generic industry template (TM Forum for Telco, ACORD for Insurance, FHIR for Healthcare, ARTS for Retail) and then spend close to a year trimming, renaming, and rewiring it to fit their actual business. Neither path scales to how fast modern data products need to ship.

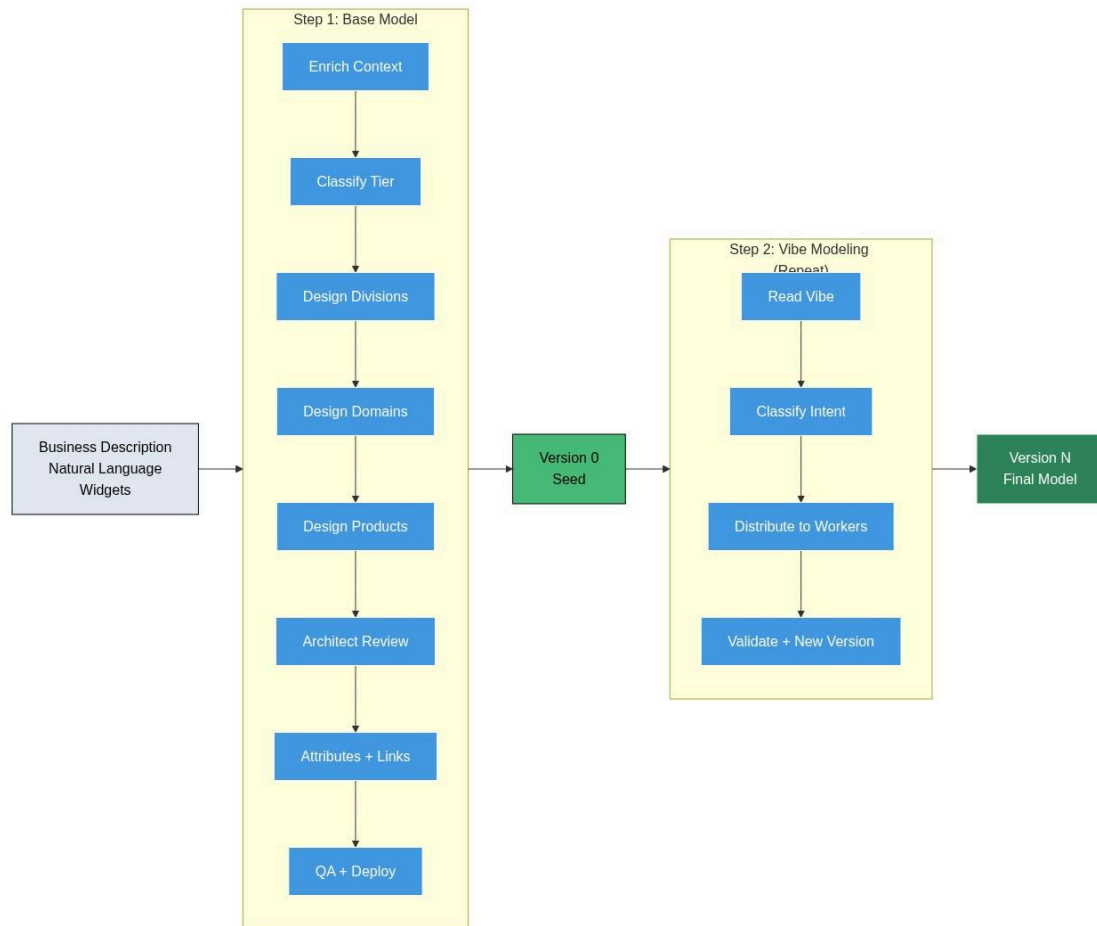
We built Vibe Data Modeling to compress that work into a single afternoon.

Today, we are announcing Vibe Data Modeling

Vibe Data Modeling is a multi-model LLM agent that produces a complete, governed, deployable Silver-layer data model from a natural-language description of your business. It ships as a single notebook in the reference repository. Four widgets, one run, a fully deployed model in Unity Catalog. If you don't like what came out, you "vibe" it in natural language until it fits your organization.

Headline facts:

- **Hours, not months.** A typical end-to-end run produces a fully deployed Minimum Viable Model in under two hours, and an Expanded Coverage Model in a single afternoon.
- **More than two hundred enforceable quality rules** across fifteen quality gates, covering naming conventions, foreign-key integrity, and attribute classification, with auto-remediation when a violation is detected.
- **Multi-model LLM ensemble.** Claude Opus and Sonnet for design, Databricks GPT-OSS for ensembling, with a judge that scores variants against one consistent rubric.
- **Two architect-review passes.** A Domain Architect reviews each domain in isolation. A Global Architect reviews the model end-to-end. Both are LLM personas with explicit principal-architect and SME instructions.
- **Native Unity Catalog deployment.** Schemas, tables, foreign keys, classification tags, metric views, RDFS knowledge graph, DBML diagram, and synthetic sample data are all generated together, and all versioned together.
- **Logical / physical separation.** Deploy one logical model in any cataloging convention. Switch conventions without rebuilding the model.



Step one produces the first complete model from your business description. Step two iterates on it through natural-language vibes. Every vibe produces a new numbered version.

How a vibe becomes a data model

Behind the four widgets, the agent runs a nineteen-stage pipeline. Three of those stages are the load-bearing ones, and they are what make the output trustworthy.

Multi-model judge ensemble. The agent does not call one LLM and accept its output. For every major design decision, it solicits proposals from multiple frontier models, scores them against a consistent rubric, and a judge model resolves the tie. This is how we keep the model deterministic-enough for an enterprise while still benefiting from each model's strengths.

Two-pass architect review. After the ensemble produces a draft, a Domain Architect persona reviews each domain on its own. It checks that the bounded context is coherent, that the FKs make business sense, and that the attributes match how the data is actually used. Then a Global Architect persona reviews the whole model end-to-end. It checks for cross-domain duplicates, single-source-of-truth violations, and structural integrity. Findings get applied automatically, with every mutation tracked.

Two hundred-plus enforceable rules. Naming, typing, classification, FK direction, FK target

existence, cycle detection, silo detection, denormalization detection, metric-view contract conformance. Each one is caught either as the model is being built or during the install gate, with auto-remediation when the fix is mechanical.

The pipeline yields a model that is consistent in shape across runs, auditable in every change, and grounded in the same architectural principles a senior data architect would apply by hand. Just compressed into a single afternoon.

What you get from one run

Every model the agent produces follows the same four-level shape: organization → divisions → domains → subdomains → products → attributes. The hierarchy is enforced by the rule catalog and by both architect-review passes, which is what makes the output predictable across very different industries.

What lands in your workspace at the end of a run:

- A logical model (model.json) with every domain, product, attribute, foreign key, and classification tag.
- A physical deployment in Unity Catalog: schemas, tables, foreign keys (informational), and classification tags applied.
- A set of metric views: reusable KPI definitions on top of the physical tables, ready for AI/BI dashboards and Genie.
- An RDFS knowledge graph for semantic tools and AI agents.
- A DBML file for visualization in dbdiagram.io or any DBML-compatible tool.
- Synthetic sample data, generated against the same model, for development and demos.
- A complete pipeline log of every decision the agent made, every rule that fired, and every mutation that landed.

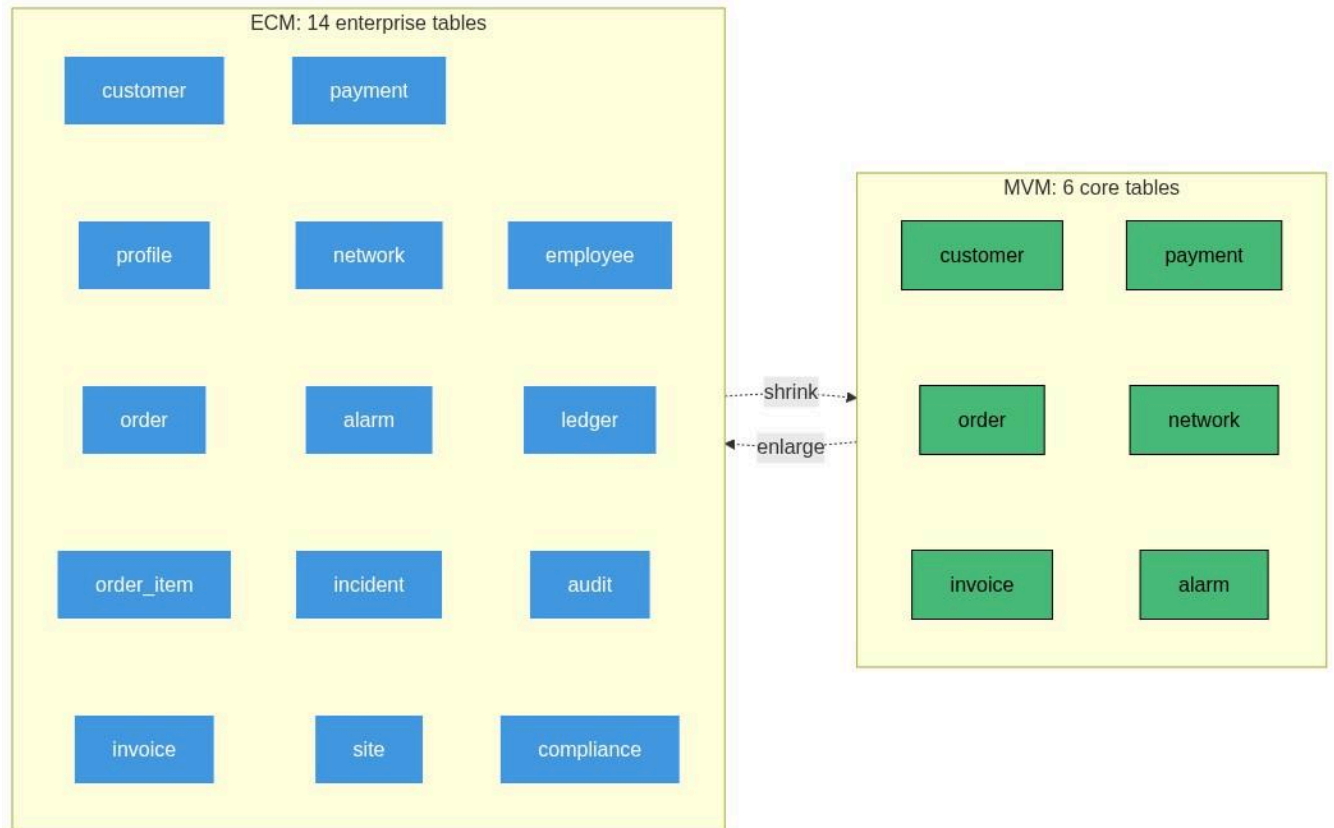
Two scopes: MVM and ECM

Most teams do not need every domain on day one. Vibe Data Modeling produces two complementary scopes.

MVM (Minimum Viable Model) covers the lean set of domains that most organizations need to start delivering analytics value. About thirty to fifty percent of the table count of an ECM, with the same attribute depth and the same quality gates. Built for SMB engagements, pilots, and early stages of a Fortune 500 deployment.

ECM (Expanded Coverage Model) covers the full enterprise breadth, including supporting domains an MVM intentionally leaves out. Built for organizations that want a single deployment to cover analytics across the whole business.

You can produce one or the other, both, or shrink an ECM to an MVM, or enlarge an MVM into an ECM. All of it from the same agent.



MVM and ECM are not different products. They are different scopes of the same model, governed by the same rules and architect reviews. Move between them with a single widget change.

Vibe it until it fits

The first run lands a complete model. Most users want to refine it, and refinement is where Vibe Data Modeling earns its name.

You write the changes you want, in plain English, into a vibes widget:

"Merge service_request and work_order into service_order. Connect procurement to product through supplier_product. Add a loyalty domain with program, tier, reward, and enrollment. Don't touch billing."

The agent reads the vibes, parses them into structured directives, and runs the pipeline again, this time with the previous model as the starting point. The same rule catalog runs. The same architect reviews run. Every change you asked for is applied if it passes validation, and rejected with an explanation if it would violate a quality gate. The output is a new numbered version. The previous version is preserved untouched.

This matters more than it sounds. It means that every iteration is auditable and reversible. It means the model can evolve with the business, accommodating a new domain, a renamed product, or a re-classified attribute, without ever rolling back to a backup. It means a

non-technical product owner can express what they want in their own language and let the agent decide how to make it real on the catalog side.



Every vibe produces a new numbered version. The model evolves forward, never sideways.

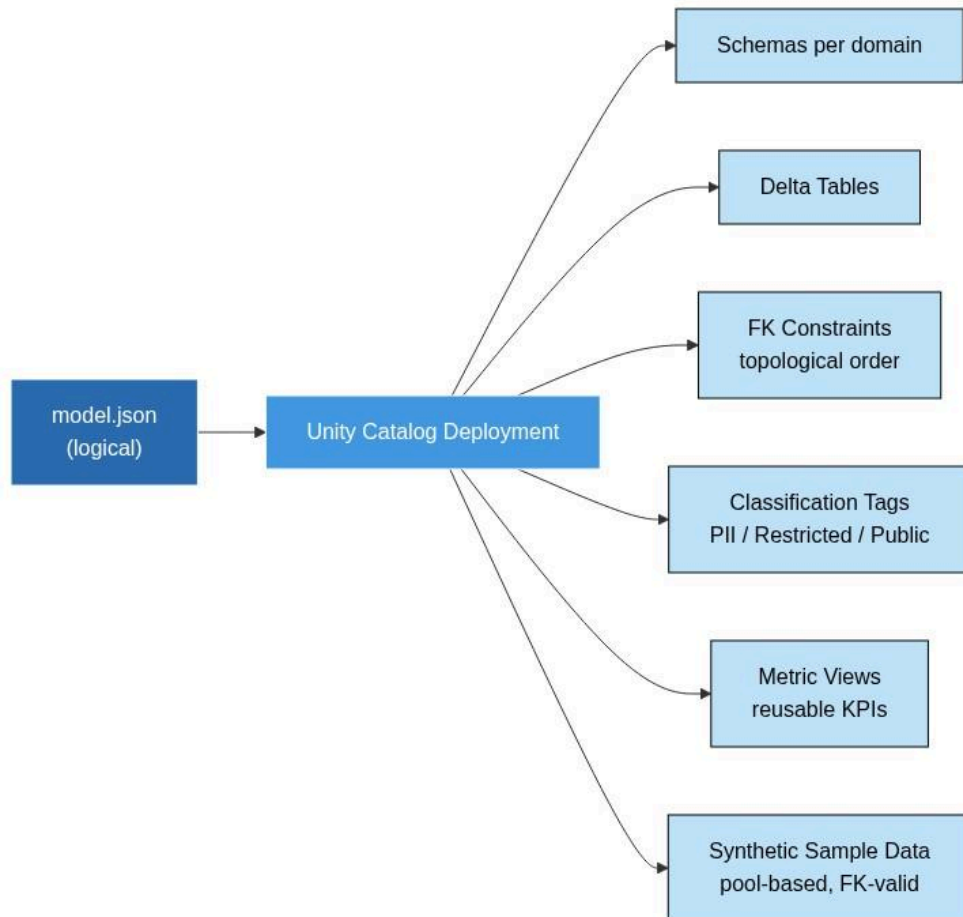
One logical model, many physical layouts

This is the feature that makes Vibe Data Modeling work identically for a mid-size SaaS company and a globally federated bank.

The logical model is one artifact: domains, products, attributes, foreign keys, classifications. The physical deployment is a separate decision controlled by a single widget. The same logical model can be rendered as:

- **One Catalog.** The entire business in one catalog; domains become schemas. Best when you have a single governance boundary.
- **Catalog per Division.** Operations, Business, and Corporate each get their own catalog; domains become schemas inside. Best when divisions are governed separately.
- **Catalog per Domain.** Every domain gets its own catalog; subdomains become schemas. Best for federated organizations where each domain owns its own governance.

If your governance reality changes, whether from a regulatory split, an acquisition, or a re-org, you redeploy to a different convention. The logical model is unchanged. The relationships, classifications, and metric views all carry over.

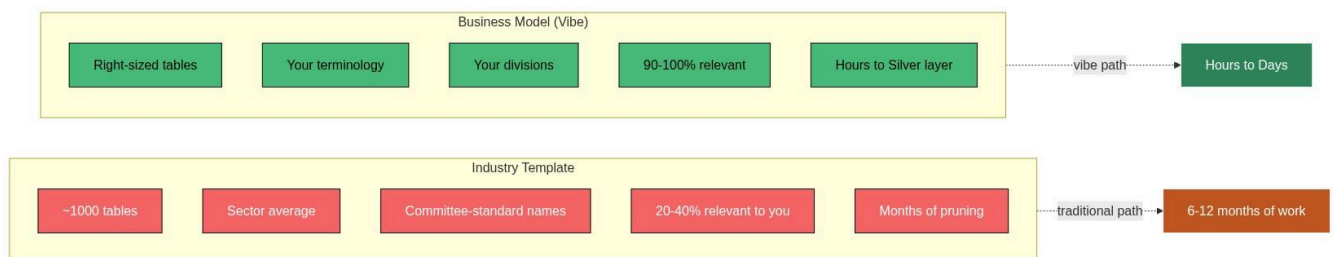


One logical model, three valid physical layouts. Switch conventions without rebuilding.

Industry templates are obsolete

The argument for buying a generic industry template was always that it gave you a head start. The reality, learned the hard way by enough customers, is that the head start costs nine to twelve months of fitting and renaming.

A generic template is the statistical average of every business in a sector. It is, by construction, nobody's actual business. Vibe Data Modeling produces a model that uses your terminology, your divisions, your domains, your operational reality. Generated in hours, deployed end to end, validated by the same rules every other Vibe model is validated against.



An industry template is the average of every business in a sector. A vibe-generated business

model is specific to one organization.

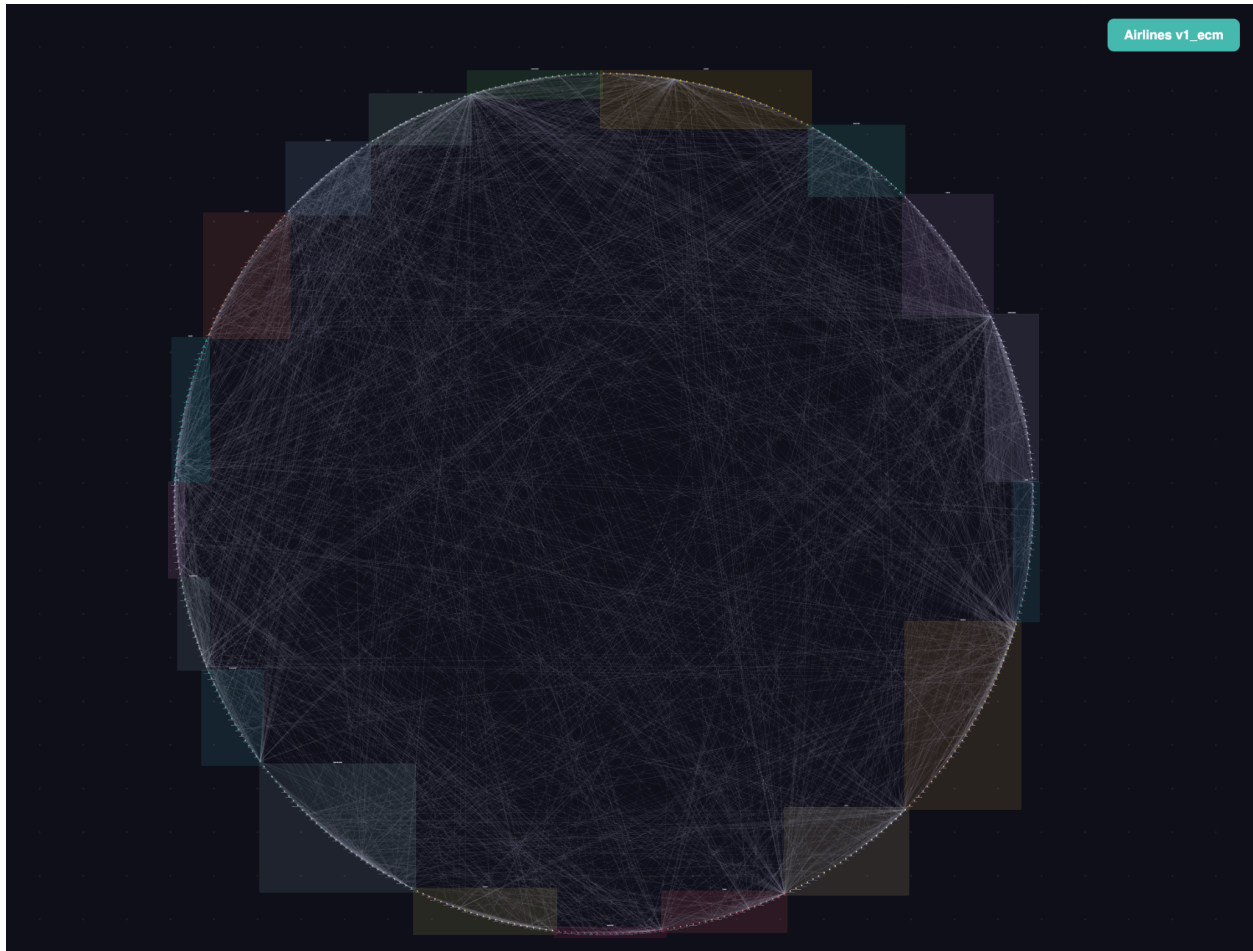
Example Viced Model - Airline Data Model

We generated an ECM data model for Airlines that covers all business divisions. Below are the model metrics:

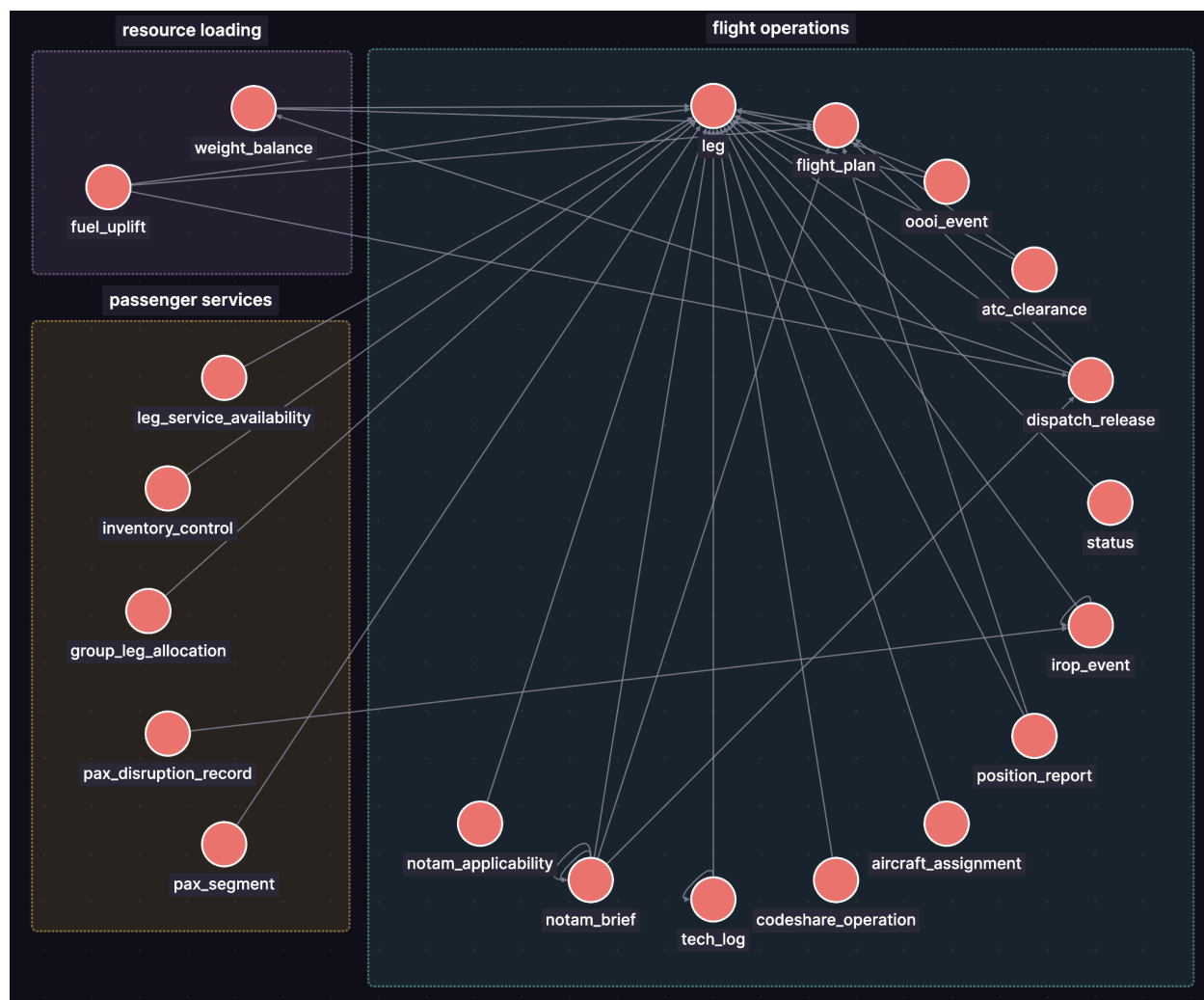
Model Metrics

Metric	Value
Model Scope	ECM (Expanded Coverage Model)
Total Domains	19
Total Subdomains	60
Total Products	420
Total Attributes	17278
Primary Keys	420
Foreign Keys	2877
Avg Attributes/Product	41.1
Metric Views	203

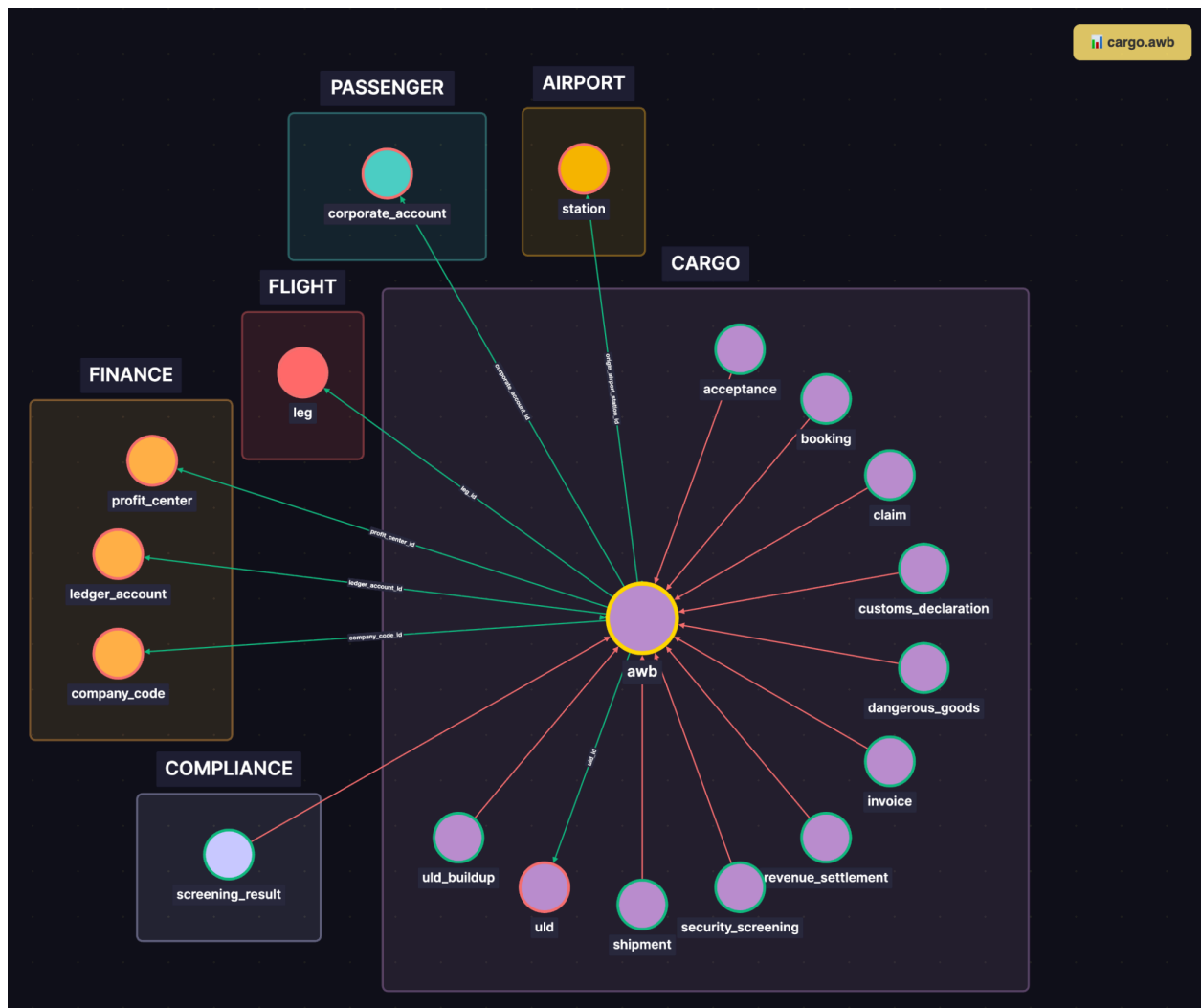
When you visualize this model, it will look like a fully connected DAG (Directed Acyclic Graph) like below:



And this is how the flight domain looks:



When you drill down to, say, the Airway bill data product (awb), it looks like below:



Available today

The reference implementation is a single Databricks notebook at `agent/dbx_vibe_modelling_agent.ipynb` in the reference repository. Fill in four core widgets (business name, business description, model scope, deployment catalog) and run. Everything else picks a sensible default based on your industry.

For producing both MVM and ECM in one go with a staged install test, use the orchestrator notebook at `runner/vibe_runner.ipynb`.

To go deeper:

- **The Vibe Data Modeling whitepaper** (available as an ebook from the Databricks Resource Library) is the full technical treatment of every pipeline stage, the complete rule catalog, the architect-review methodology, and the LLM ensemble architecture.
- **The Architecture Center landing page** is the industry-agnostic reference architecture, with dedicated pages for Financial Services, Healthcare, Retail, Manufacturing, Telecommunications, Energy, Public Sector, and Media.
- **The reference repository** at `github.com/databricks/vibe-modelling-agent` contains the agent notebook, the orchestrator runner, a test harness, and a full set of

guides covering design, integration, quality gates, and the rule catalog.

If your team has been carrying a Silver-layer project for months without shipping, this is the shortest path we have found to actually shipping one. Describe your business in plain English. Get a model. Iterate it until it fits. Move on.